

Threads in Python

Quick Review





The main idea of this class is a hand-on a section to learning programming **Thread** and **Multi-Processing** with Python in a practical fashion.



Threads x Multi-Threads

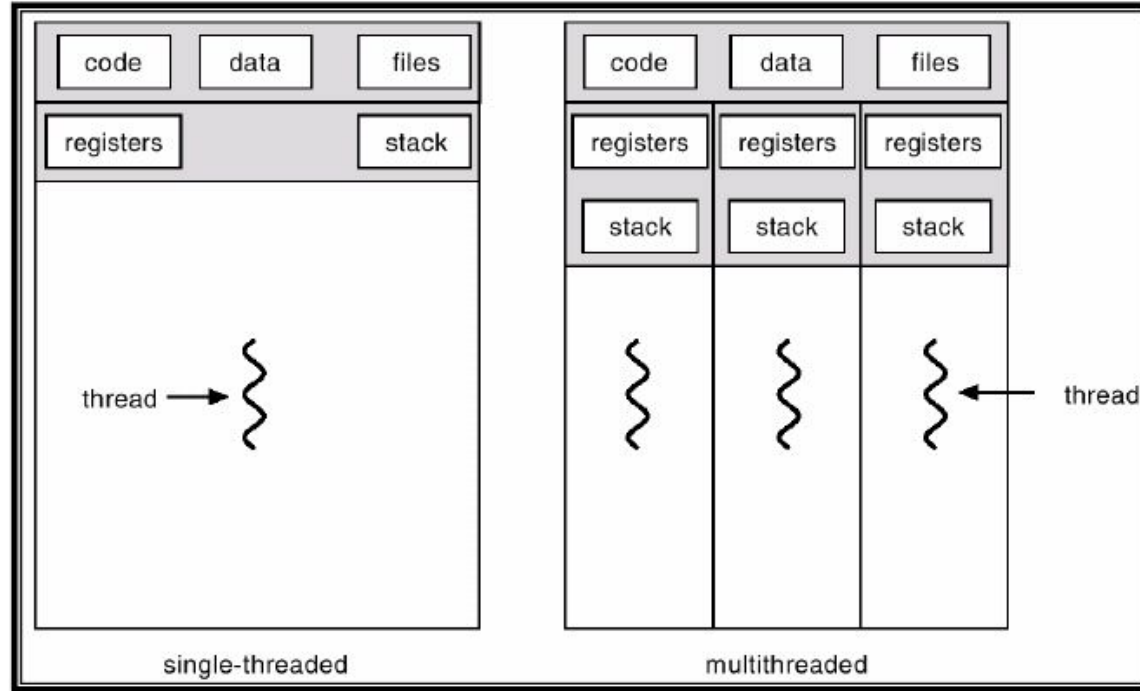


Image: <http://www.pling.org.uk/cs/opsimg/threadmodel.png>

MultiThreads x MultiProcesses

Multiprocessing

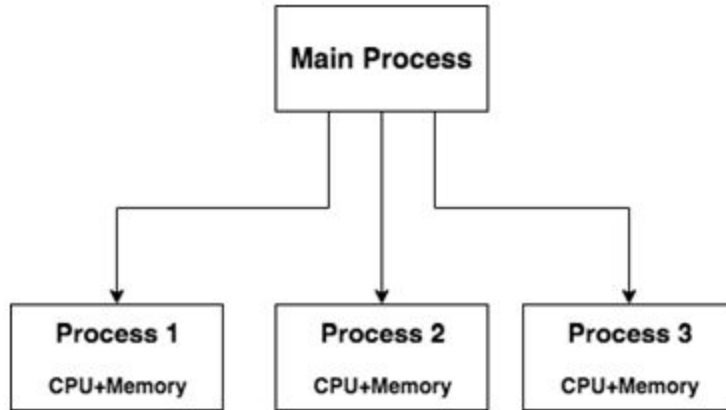
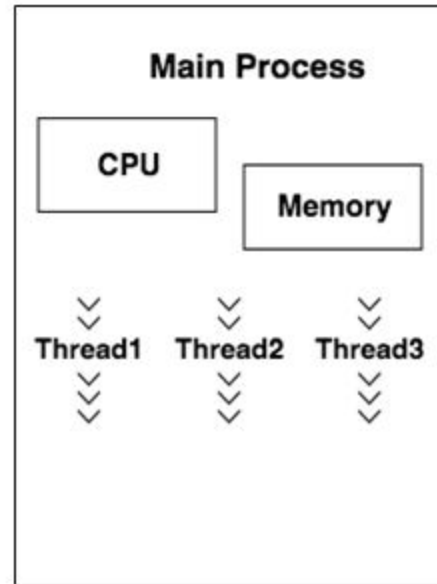


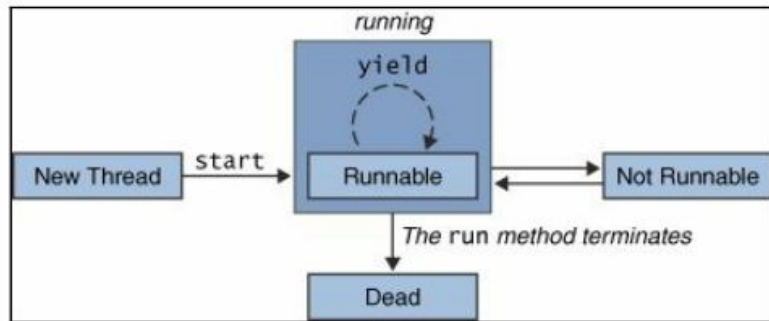
Image: https://datanoon.com/blog/multiprocessing_in_python/

Multithreading



State flow chart

The following diagram represents the five different states that a thread can be in as well as the possible transitions from one state to another:



Source: <http://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/lifecycle.html>

References

MULTIPROCESSING IN PYTHON

APRIL 6, 2019

With increasing number of power hungry applications, the demand for speed and low latency has become a challenge in certain situations. However, the availability of machines with multiple processors/processes with multiple cores help us combat such situations. This post would guide you through using multiprocessing in python.

Introduction

In contemporary times, a lot of CPUs are being manufactured with multiple cores to boost performance by enabling parallelism and concurrency of applications. Consequently, such CPUs can be leveraged to write code which can run in parallel thereby speeding up your application. This is where the Multiprocessing package in python comes into play. This API is very similar to the threading module in Python.

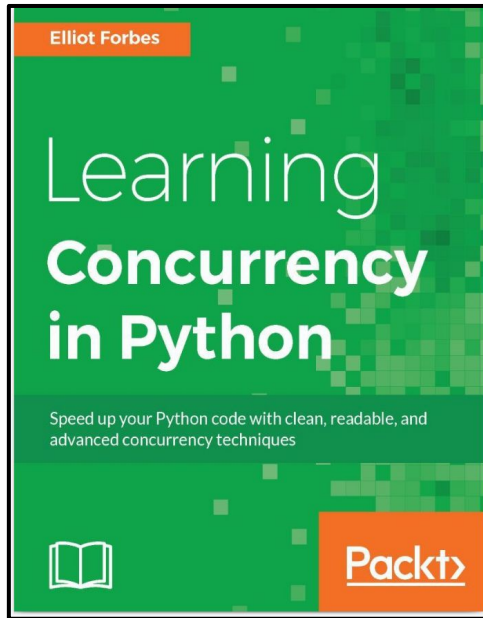
The previous post on [Multithreading in Python](#) provides a clear explanation on the threading module, [click here](#) to read through the same if you haven't.

Table of Contents

- [What is Multiprocessing](#)
- [Multiprocessing Example](#)
- [Multiprocessing Use Cases](#)
- [Multiprocessing vs Multithreading](#)
- [Multiprocessing Pool](#)
- [Multiprocessing vs Serial Processing Performance Test](#)
- [Sharing Data Between Processes](#)
- [Multiprocessing Queues](#)
- [Multiprocessing Value and Lock](#)
- [Multiprocessing Manager](#)
- [Conclusion](#)

- https://datanoon.com/blog/multiprocessing_in_python/

Book indication (this class is based on this book)



shorturl.at/abhqL

Some Advantages of Threading

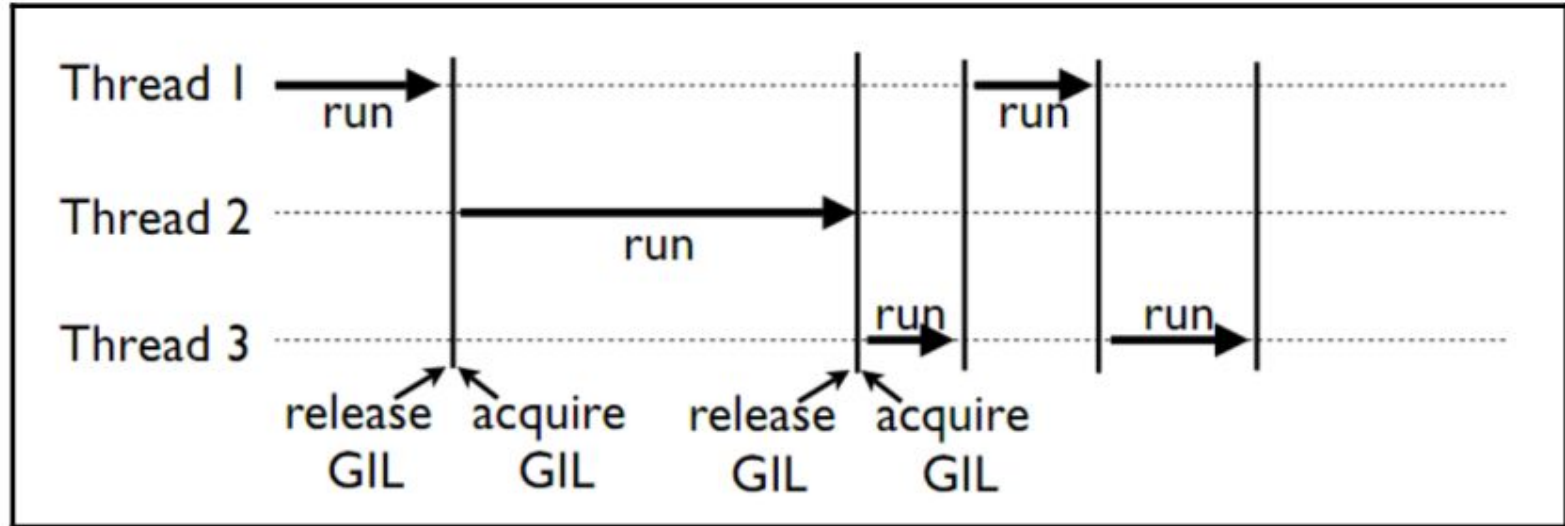
- Multiple threads are excellent for speeding up blocking I/O bound programs.
- They are lightweight in terms of memory footprint when compared to processes.
- Threads share resources, and thus communication between them is easier.

Some disadvantages Pythons and Threads

- CPython threads are hamstrung by the limitations of the global interpreter lock (GIL).
- While communication between threads may be easier, you must be very careful not to implement code that is subject to race conditions.
- It's computationally expensive to switch context between multiple threads. By adding multiple threads, you could see a degradation in your program's overall performance.

The limitations of Python

The **GIL** is essentially a mutual exclusion lock which prevents multiple threads from executing Python code in parallel.



IO Programs

The next example we will download 10 images. The first version is a sequential program, and the second one is a concurrent version. The download is a good example of IO-block instruction. Does the second version better?

Benefits of Concurrency on IO-Block programs

```
import urllib.request
```

```
import time
```

[Sequential Logic]

```
def downloadImage(imagePath, fileName):
```

```
    print("Downloading Image from ", imagePath)
```

```
    urllib.request.urlretrieve(imagePath, fileName)
```

```
def main():
```

```
    for i in range(10):
```

```
        imageName = "temp/image-" + str(i) + ".jpg"
```

```
        downloadImage("https://picsum.photos/200/300?random=1", imageName)
```

```
if __name__ == '__main__':
```

```
    t1 = time.time()
```

```
    main()
```

```
    print("Spent time:", time.time() - t1)
```

Benefits of Concurrency on IO-Block programs

```
python /home/rayner/PycharmProjects/pythonThreads/sequential_download.py
```

Downloading Image from <https://picsum.photos/200/300?random=1>

Downloading Image from <https://picsum.photos/200/300?random=1>

(...)

Downloading Image from <https://picsum.photos/200/300?random=1>

Downloading Image from <https://picsum.photos/200/300?random=1>

Spent time: 10.812902450561523s

How do many CPU we have?

```
1 import multiprocessing
2
3 if __name__ == '__main__':
4     count = multiprocessing.cpu_count()
5     print(count)
```

Run:  count_number_cpu x

  /home/rayner/PycharmProjects/python
 8


  Process finished with exit code 0
 

Benefits of Concurrency on IO-Block programs

```
1  import threading
2  import urllib.request
3  import time
4
5  def downloadImage(imagePath, fileName):
6      print("Downloading Image from ", imagePath)
7      urllib.request.urlretrieve(imagePath, fileName)
8      print("Completed Download")
9
10 def executeThread(i):
11     imageName = "temp/image-" + str(i) + ".jpg"
12     downloadImage("http://lorempixel.com/400/200/sports", imageName)
13
```

Benefits of Concurrency on IO-Block programs

```
14 def main():
15     t0 = time.time()
16     threads = []
17     for i in range(10):
18         thread = threading.Thread(target=executeThread, args=(i,))
19         threads.append(thread)
20         thread.start()
21     for i in threads:
22         i.join()
23     t1 = time.time()
24     totalTime = t1 - t0
25     print("Total Execution Time {}".format(totalTime))
26
27 if __name__ == '__main__':
28     main()
```

Benefits of Concurrency on IO-Block programs

python concurrency_download.py

Downloading Image from <https://picsum.photos/200/300?random=1>

(...)

Completed Download

Completed Download

Completed Download

Total Execution Time 3.348254680633545s

Sequential prime factorization

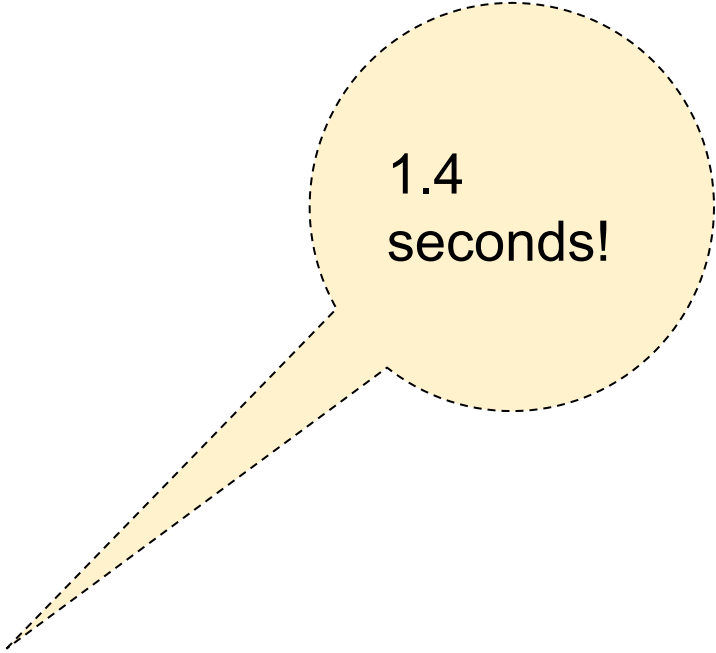
```
1  import time
2  import random
3
4  def calculatePrimeFactors(n):
5      primfac = []
6      d = 2
7      while d*d <= n:
8          while (n % d) == 0:
9              primfac.append(d)
10             n //= d
11             d += 1
12         if n > 1:
13             primfac.append(n)
14     return primfac
```

```
15
16 def main():
17     t0 = time.time()
18     for i in range(10000):
19         rand = random.randint(20000, 100000000)
20         print(calculatePrimeFactors(rand))
21     t1 = time.time()
22     totalTime = t1 - t0
23     print("Execution Time: {}".format(totalTime))
24
25 if __name__ == '__main__':
26     main()
```

Sequential prime factorization

[46349561]
[7, 4607191]
[31, 2903]
[2, 5, 7, 7, 40099]
[2, 2, 3, 17, 19, 10501]
(...)
[227, 74449]
[29, 769, 3709]
[3, 5818559]
[19, 571, 8059]
[3, 3, 131, 31193]
[17, 53, 71917]

Execution Time: 1.408031940460205s



1.4
seconds!

Concurrent prime factorization

```
1 import time
2 import random
3 from multiprocessing import Process
4
5 def calculatePrimeFactors(n):
6     primfac = []
7     d = 2
8     while d*d <= n:
9         while (n % d) == 0:
10             primfac.append(d)
11             n //= d
12         d += 1
13     if n > 1:
14         primfac.append(n)
15     return primfac
16
```

```
17 def executeProc():
18     for i in range(1000):
19         rand = random.randint(20000, 1000000000)
20         print(calculatePrimeFactors(rand))
21
22 def main():
23     print("Starting number crunching")
24     t0 = time.time()
25     procs = []
26     for i in range(10):
27         proc = Process(target=executeProc, args=())
28         procs.append(proc)
29         proc.start()
30     for proc in procs:
31         proc.join()
32     t1 = time.time()
33     totalTime = t1 - t0
34     print("Execution Time: {}".format(totalTime))
```

Parallelism x Concurrency

Concurrency and parallelism are two concepts that are commonly confused. The reality, though, is that they are quite different, and if you designed software to be concurrent when instead you needed parallel execution, then you could be seriously impacting your software's true performance potential.

Some important points to see

- What is concurrency and what are the major bottlenecks that impact our applications.
- What is parallelism and how does this differ from concurrency.
- The different styles of computer system architecture and how we can utilize these effectively, using either concurrency or parallelism.
- A brief overview of computer memory architecture.

Parallel prime factorization

```
35  
36 ► if __name__ == '__main__':  
37     main()
```

(...)

[31550803]

[3, 3, 7, 17, 75659]

[2, 23906287]

[2, 3, 17, 101, 7933]

[5, 43, 53639]

[72459671]

[2, 29, 974749]

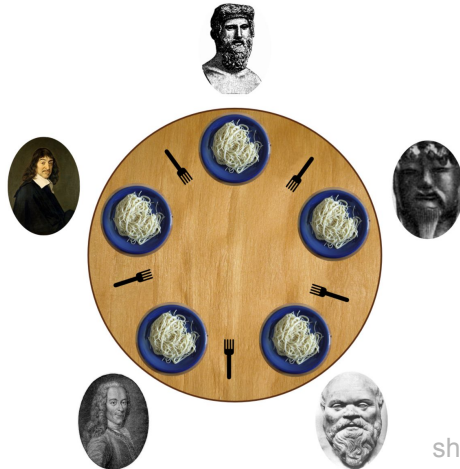
[2, 2, 2, 5, 5, 259151]

0.5
seconds!

Execution Time: 0.47304320335388184

What is concurrency

Concurrency is, essentially, the practice of doing multiple things at the same time, but not, specifically, in parallel. It can help us to improve the perceived performance of our applications, and it can also improve the speed at which our applications run.



shorturl.at/lqxVZ

Random output example

```
1 import random
2 import threading
3 import time
4 counter = 1
5
6 def workerA():
7     global counter
8     while counter < 1000:
9         counter += 1
10        print("Worker A is incrementing counter to {}".format(counter))
11        sleepTime = random.randint(0, 1)
12        time.sleep(sleepTime)
13
14 def workerB():
15     global counter
16     while counter > -1000:
17         counter -= 1
18        print("Worker B is decrementing counter to {}".format(counter))
19        sleepTime = random.randint(0, 1)
20        time.sleep(sleepTime)
21
```

```
22
23 def main():
24     t0 = time.time()
25     thread1 = threading.Thread(target=workerA)
26     thread2 = threading.Thread(target=workerB)
27     thread1.start()
28     thread2.start()
29     thread1.join()
30     thread2.join()
31     t1 = time.time()
32     print("Execution Time {}".format(t1 - t0))
33
34
35 if __name__ == '__main__':
36     main()
```

Random output example

Look at the A and
B output
sequence has
not logic order!

Worker A is incrementing counter to 3

Worker A is incrementing counter to 4

Worker A is incrementing counter to 5

Worker A is incrementing counter to 6

Worker A is incrementing counter to 7

Worker B is decrementing counter to 6

Worker A is incrementing counter to 7

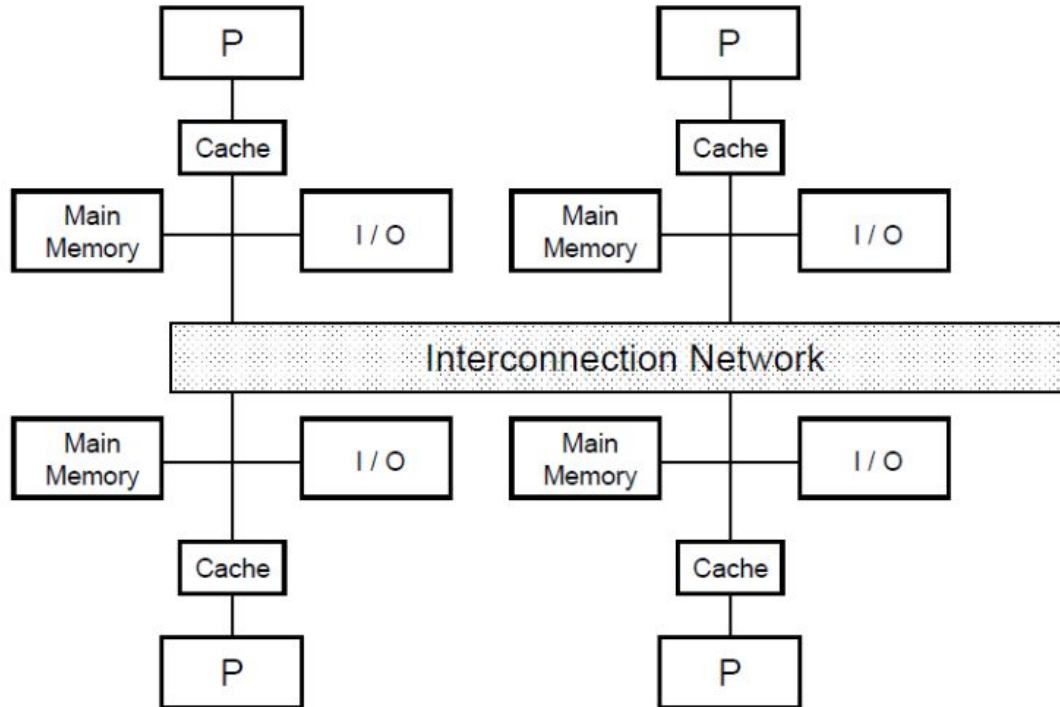
Worker A is incrementing counter to 8

Worker B is decrementing counter to 7

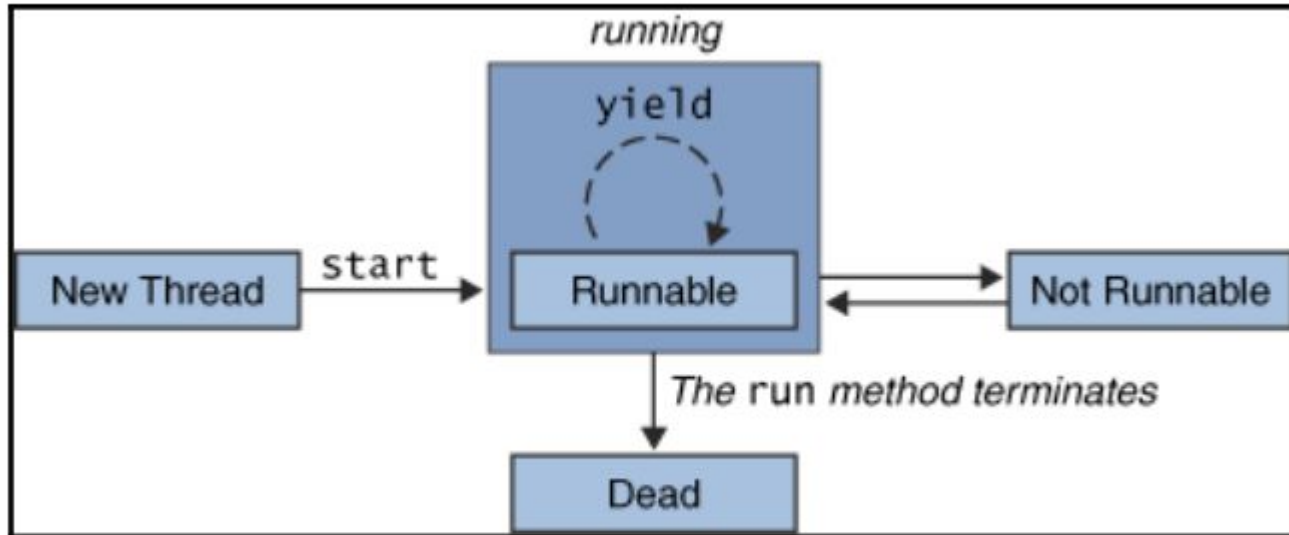
Worker A is incrementing counter to 8

Worker A is incrementing counter to 9

Non-uniform Memory Access - NUMA



Thread States flowchart



Source: <http://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/lifecycle.html>

Thread Class

```
1  from threading import Thread
2  import time
3
4  class myWorkerThread(Thread):
5      def __init__(self):
6          print("Hello world")
7          Thread.__init__(self)
8
9      def run(self):
10         print("Thread is now running")
11         print("My name:", self.name)
12         print("Thread is now sleeping")
13         time.sleep(1)
14
```



A diagram consisting of a rectangular box with a downward-pointing arrow on its top edge. The arrow originates from the right side of the box and points towards the first line of code in the adjacent code block, indicating the flow of execution from the class definition to its instantiation.

```
15  myThread = myWorkerThread()
16  print("Created my Thread Object")
17  myThread.start()
18  print("Started my thread")
19  myThread.join()
20  print("My Thread finished")
```

Forking

```
1 import os
2
3 def child():
4     print("We are in the child process with PID= %d" % os.getpid())
5
6
7 def parent():
8     print("We are in the parent process with PID= %d" % os.getpid())
9     newRef = os.fork()
10    if newRef == 0:
11        child()
12    else:
13        print("We are in the parent process and our child process has PID= %d" % newRef)
14
15 if __name__ == '__main__':
16     parent()
17
```

Daemonizing a thread

Firstly, before we look at daemon threads, I feel it is important to know what these are. Daemon threads are 'essentially' threads that have no defined endpoint. They will continue to run forever until your program quits.

Daemonizing a thread

```
1  import threading
2  import time
3
4  def standardThread():
5      print("Starting my Standard Thread")
6      time.sleep(20)
7      print("Ending my standard thread")
8
9
10 def daemonThread():
11     while True:
12         print("Sending Out Heartbeat Signal")
13         time.sleep(2)
14
15 if __name__ == '__main__':
16     standardThread = threading.Thread(target=standardThread)
17     daemonThread = threading.Thread(target=daemonThread)
18     daemonThread.setDaemon(True)
19     daemonThread.start()
20     standardThread.start()
```

You'll notice that we also use the `setDaemon` function on our **daemonThread** object. This simply sets the thread object's daemon flag to whatever we pass into this function, and is only really used for reference.

Get Current Thread

```
1  import threading
2
3  def threadTarget():
4      print("Current Thread: {}".format(threading.current_thread()))
5
6      threads = []
7  for i in range(10):
8      thread = threading.Thread(target=threadTarget)
9      thread.start()
10     threads.append(thread)
11
12     for thread in threads:
13         thread.join()
```

Main Thread

```
1 import threading
2 import time
3
4 def myChild_or_MainThread():
5     print("Child Thread Starting")
6     time.sleep(5)
7     print("Current Thread -----")
8     print(threading.current_thread())
9     print("-----")
10    print("Main Thread -----")
11    if threading.current_thread() == threading.main_thread():
12        print("I was called by main")
13    else:
14        print("I was called by child")
15    print("-----")
16    print("Child Thread Ending")
17
18 child = threading.Thread(target=myChild_or_MainThread)
19 myChild_or_MainThread()
20 child.start()
21 child.join()
```